Chapter 04

HAProxy: Edge Proxy & SSL Termination

One IP. One certificate. Every request in the lab flows through this box first — this chapter builds the single entry point that makes the whole platform reachable, encrypted, and observable.

11	443/80	5yr	7
ROUTED BACKENDS	PORTS TERMINATED	CERT VALIDITY	HARDENING CONTROLS

The Edge Proxy Problem

12 services. 12 VMs. 12 different IPs. How does a user reach GitLab without memorizing 10.0.2.14?

❌ Bad Approach

```
Remember the IP. Type https://10.0.2.14/
```

- ✗ IPs change (VM migration, rebuilds)
- ✗ Not human-friendly
- ✗ No way to enforce company-wide policies (TLS, rate limiting, logging)
- ✗ No way to switch backend servers without changing client configuration

✔️ Good Approach — Use an Edge Proxy

- ✓ Single memorable hostname (gitlab.devsecops.com)
- ✓ IPs can change freely behind the proxy
- ✓ Central policy enforcement: TLS, rate limits, logging
- ✓ Backends swap transparently — clients never notice



🏗️ Analogy — The Hotel Concierge Desk

Guests (clients) ask the concierge for a room ("I want GitLab"). The concierge routes them to the correct room (backend server) without the guest ever knowing which floor or building it's on. If a room is under maintenance, the concierge quietly routes to a backup — the guest never notices.

4.1 CAPABILITIES

What HAProxy Does

1

Single Entry Point

Clients connect to one IP/hostname — sec-edge-01

2

TLS Termination

Encrypts client-to-proxy traffic; decrypts internally

3

Host-Based Routing

Routes gitlab.devsecops.com → GitLab backend

4

Load Balancing

Spreads traffic across multiple backends

5

Failover

If a backend is down, requests go to healthy backends

6

Traffic Policies

Rate limiting, request filtering, header manipulation

7

Comprehensive Logging

Every request visible for audit and debugging

4.2 PROXY LANDSCAPE

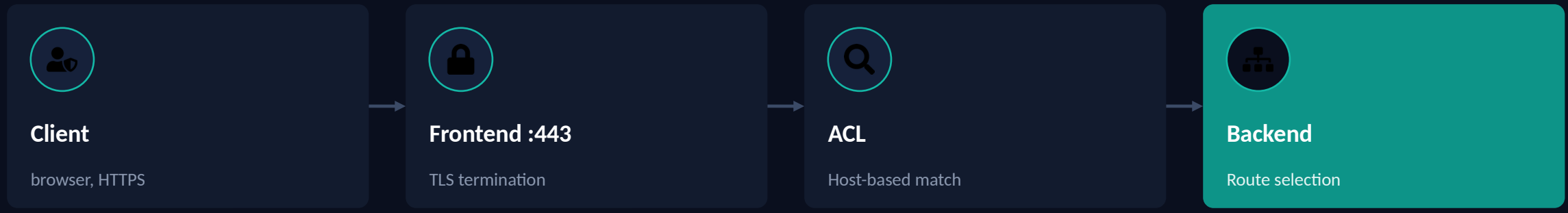
HAProxy vs Nginx vs Other Proxies

Proxy	Strength	Best For	Lab Role
HAProxy	Layer 4 + 7 load balancing, fast, battle-tested	TCP/HTTP routing, failover	Primary edge proxy
Nginx	Layer 7 HTTP routing, elegant config, dynamic modules	Web server + reverse proxy	Secondary proxy / app server
Traefik	Kubernetes-native, dynamic config, modern	Kubernetes services	*Not used (K8s native in Phase 5)*

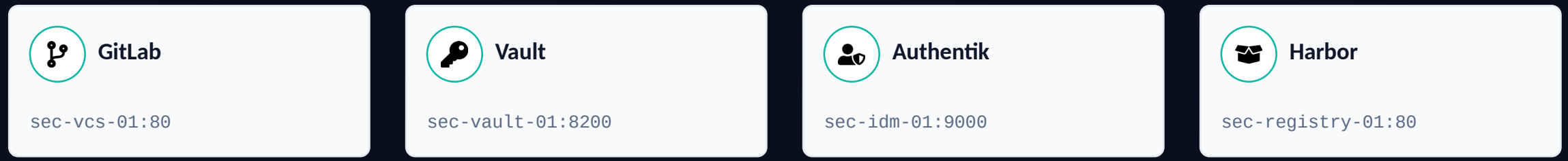
In this lab:

HAProxy is the primary edge proxy (TLS, routing, load balancing). Nginx runs behind it as a secondary proxy for web server duties (Chapter 05) — the two work together in a typical production architecture.

How a Request Flows Through the Edge



sec-edge-01 (10.0.2.12) — HAProxy + Nginx



Dashed routes in the book diagram (Vault, Auth, Harbor) work the same way — one ACL, one backend, one server pool per service.

Key HAProxy Concepts

Concept	Definition	Lab Example
Frontend	What clients connect to (IP:port, TLS settings)	*Port 443, HTTPS, certificates*
Backend	Pool of servers that handle requests	*GitLab servers, Vault servers, etc.*
ACL	Condition matching on request data	*If hostname = gitlab.devsecops.com*
Route	Decision rule connecting ACLs to backends	*If ACL matches, use this backend*
Health Check	Verify backend is alive before sending traffic	*TCP connect, HTTP GET, etc.*
Session Stickiness	Route same client to same backend (for state)	*For apps that cache client state*
Rate Limiting	Limit requests per second/minute	*Prevent DDoS, API abuse*

4.4 SETUP

Installation & Verification

On sec-edge-01:

```
$ ssh sec-edge-01

sudo apt update
sudo apt install -y haproxy ssl-cert curl
```

Verify:

```
$ haproxy -v
→ HA-Proxy version 2.8.x  2023/11/16 ...
```

WHAT EACH PACKAGE DOES

- **`haproxy`**

  The edge proxy daemon

- **`ssl-cert`**

  Utilities for managing self-signed SSL certificates

- **`curl`**

  Test tool for verifying proxy behavior

💡 **Key Takeaway**

HAProxy ships in Ubuntu's default repos — no third-party PPA needed to get a production-grade edge proxy.

Self-Signed Wildcard Certificates

One certificate for *.devsecops.com covers every subdomain the edge will ever route.

1 Create the cert directory

```
sudo mkdir -p /etc/haproxy/certs
sudo chmod 700 /etc/haproxy/certs
```

2 Generate the wildcard cert

```
sudo openssl req -x509 -newkey rsa:2048 \
  -keyout devsecops.com.key \
  -out devsecops.com.crt -days 1825 -nodes \
  -subj "/CN=*.devsecops.com/O=DevSecOps Lab/C=US"
```

3 Combine into one PEM

```
cat devsecops.com.key devsecops.com.crt \
  | tee devsecops.com.pem
chmod 600 devsecops.com.pem
```

Verify the certificate

```
$ openssl x509 -in devsecops.com.crt \
    -text -noout | head -20
→ Subject: CN = *.devsecops.com
→ Not Before: Jan 1 2026 GMT
→ Not After : Jan 1 2031 GMT
→ Public-Key: (2048 bit)
```



🔐 Security Note

This certificate is self-signed — not trusted by browsers. You'll get a warning when visiting in a browser. That's expected in a lab. In production, use Let's Encrypt or your corporate CA.

Configuration Anatomy — Five Sections

1. Global	Process-wide settings — logging, chroot, user/group, TLS ciphers, tuning
2. Defaults	Applied to all frontends/backends — mode, timeouts, error pages
3. Frontend	What clients connect to — bind 443/80, ACLs, use_backend rules
4. Backend	Pools of servers — 11 backends, one per lab service, with health checks
5. Listen (stats)	Combined frontend+backend for the real-time statistics page

💡 **Key settings worth knowing:** `ssl-min-ver TLSv1.2` · `no-tls-tickets` · `tune.ssl.default-dh-param 2048` · `timeout connect 5000` · `timeout tunnel 1h (WebSockets)`

Host-Based Routing: One ACL Per Service

The frontend matches the Host header, then use_backend hands off to the right server pool.

Hostname	Backend	Target
gitlab.devsecops.com	be_gitlab	sec-vcs-01:80
vault.devsecops.com	be_vault	sec-vault-01:8200
auth / sso.devsecops.com	be_auth	sec-idm-01:9000
ldap.devsecops.com	be_ldap	sec-idm-01:17170
harbor / registry.devsecops.com	be_harbor	sec-registry-01:80
sonar.devsecops.com	be_sonar	sec-appsec-01:9000

Hostname	Backend	Target
defectdojo / dojo.devsecops.com	be_defectdojo	sec-appsec-01:8000
nexus.devsecops.com	be_nexus	sec-registry-01:8081
kibana / elk.devsecops.com	be_kibana	sec-obs-01:5601
wazuh / siem.devsecops.com	be_wazuh	sec-siem-01:443
k8s / kubernetes.devsecops.com	be_k8s	k8s-cp-01:6443

default_backend be_default returns an HTTP deny when no ACL matches — there is no silent fallthrough.

Validate, Start & Open the Firewall

1 Validate the syntax

```
$ sudo haproxy -c -f
/etc/haproxy/haproxy.cfg
→ Configuration file is valid
```

2 Start & enable

```
$ sudo systemctl restart haproxy
$ sudo systemctl enable haproxy
$ sudo systemctl status haproxy
→ Active: active (running)
```

3 Open the firewall

```
$ sudo ufw allow from 10.0.2.0/24 to
any port 80
$ sudo ufw allow from 10.0.2.0/24 to
any port 443
→ 443/tcp  ALLOW IN  10.0.2.0/24
```

4.10 VERIFICATION

Testing HAProxy Routing

Test 1 — Local Connection

```
curl -k https://localhost/
```

403 Forbidden — no default backend (expected)

Test 2 — GitLab Routing

```
curl -k https://10.0.2.12/ -H "Host: gitlab.devsecops.com"
```

502 Bad Gateway — routing works, GitLab not installed yet

Test 3 — Health Check Logs

```
sudo tail -20 /var/log/haproxy.log
```

Server be_gitlab/gitlab-01 is DOWN (backend not running)

Test 4 — Statistics Page

```
curl http://127.0.0.1:8404/stats
```

Frontend + backend status, all showing DOWN for now

💡 **A 502 is a pass, not a failure:** HAProxy received the request, matched the ACL, and routed it correctly. The backend just isn't running yet — that comes in later chapters.

Load Balancing Across Multiple Backends

```
backend be_gitlab
  mode http
  balance roundrobin
  option httpchk GET /healthz
  server gitlab-01 10.0.2.14:80 check weight 10
  server gitlab-02 10.0.2.19:80 check weight 10
  server gitlab-03 10.0.2.25:80 check weight 5
```

In this lab we run one instance of each service, so plain roundrobin is fine — but the config scales to N servers with zero client-side changes.



`roundrobin`

Each server takes turns



`leastconn`

Send to server with fewest connections



`source`

Hash client IP to same server (sticky sessions)



`uri`

Hash request URI to same server (sticky for cache)

Security: HAProxy Hardening

Setting	What It Does	Why
`ssl-min-ver TLSv1.2`	Only allow TLS 1.2+	*Disable old insecure protocols*
`no-tls-tickets`	Don't use TLS session resumption	*Reduce attack surface*
`tune.ssl.default-dh-param 2048`	Strong Diffie-Hellman	*Prevent weak key negotiation*
`log global`	Log all requests	*Audit trail for compliance*
`tune.maxconn 4096`	Connection limit	*Prevent DoS exhaustion*
`Run as haproxy user`	Not root	*Principle of least privilege*

ADDITIONAL HARDENING (OPTIONAL, FOR FUTURE)

Rate limiting per IP · WAF rules (ModSecurity) · HTTP/2 ALPN negotiation · HSTS headers (HTTP Strict Transport Security)

Troubleshooting Quick Reference

"Address already in use" on 443	sudo lsof -i :443 — kill the conflicting process or change HAProxy's port
HAProxy won't start after a config change	sudo haproxy -c -f /etc/haproxy/haproxy.cfg — shows the exact line with the error
Backend shows DOWN in stats	Check the service is running, then test its health check path directly with curl
TLS handshake errors in logs	openssl x509 -in devsecops.com.crt -text -noout — confirm the cert is well-formed
Client gets "502 Bad Gateway"	HAProxy routed correctly; the backend isn't responding. Check service status, health path, and port.

4.17 CHAPTER SUMMARY

What We Built — And Why It Matters

- **HAProxy edge proxy**
  Single entry point for the entire lab
- **Host-based routing**
  Route traffic by hostname to backends
- **Self-signed certificates**
  Functional HTTPS for the lab
- **TLS termination**
  All HTTPS traffic encrypted to the edge
- **Health checks**
  Verify backends alive, remove unhealthy ones
- **Statistics interface**
  Monitor backend health in real time

💡 Key Takeaway — The Edge Proxy Is Your Security Checkpoint

Every request now flows through HAProxy: logged for audit, encrypted end to end, ready for rate limits and WAF rules, with services free to move internally without clients ever noticing.

Next: Chapter 06 *— oauth2-proxy sits in front of HAProxy to enforce OIDC authentication, making the edge your true security checkpoint.*